

Roadmap del Clasificador de Repositorios

Mejoras propuestas - Niveles A, B y C

De acciones inmediatas a features arquitectonicas

Para	Jorge Mera (jlmera) - DUGOTEX
Proyecto	clasificador v1.0 (Rust + eframe + egui)
Estado	157 repos ~25 categorias 5 fases completadas
Fecha	2026-05-04
Documento	Propuesta de evolucion incremental

Este documento organiza las ideas de evolucion del Clasificador en tres niveles segun urgencia y profundidad. Cada feature incluye el problema que resuelve, una solucion propuesta, un mockup o snippet de codigo y una estimacion de esfuerzo/beneficio para facilitar la priorizacion.

Contenido

Introduccion y estado actual	3
A. Acciones inmediatas (sin codigo nuevo)	4
B. Mejoras tecnicas a corto plazo (15 features)	5
B1. Productividad del editor de categorias	5
B2. Inteligencia del descubridor	7
B3. Sistema de seguimiento de repos	8
B4. Quick actions y atajos	10
B5. Mantenimiento y observabilidad	11
C. Features arquitectonicas a futuro (10 ideas)	12
C1. Multi-tag y sub-categorias	12
C2. Inteligencia con LLM	13
C3. Automatizacion y schedule	14
C4. Salud del ecosistema	15
Apendice: matriz de priorizacion	16

Introduccion y estado actual

El Clasificador de Repositorios es una aplicacion nativa Windows escrita en Rust con interfaz grafica eframe+egui. Su mision: mantener organizada una coleccion creciente de repositorios clonados desde GitHub, clasificandolos automaticamente en categorias curadas segun una mezcla de heurísticas (nombre, descripcion, manifests de stack) y metadata oficial de GitHub (descripcion del autor + topics).

Lo que ya esta construido

Fase	Entregable principal
1 - Nucleo	Scan + classify + index -> buscador.html
2 - GUI	eframe+egui 6 botones de accion log coloreado
3 - Integraciones	GitHub API (PAT) Anthropic API (DPAPI) Wiki Obsidian
3a/b/c - Categorias	data/categorias.json editable reclasificar editor visual
3d - Robustez	Escritura atomica (anti-corrupcion Syncthing)
4 - Descubridor	Auto-discovery de categorias por GitHub topics
5 - Refinamiento	Compactar Borrar vacias Preview Solo verdes

El sistema funciona, pero hay 3 niveles de mejoras pendientes que conviene abordar en orden distinto:

Nivel A	Acciones que solo requieren clicks tuyos en la app actual.
Nivel B	Mejoras incrementales de codigo - se trata de pulir UX y agregar memoria.
Nivel C	Features arquitectonicas que cambian como pensas la herramienta.

A. Acciones inmediatas (sin código nuevo)

Estas 5 acciones limpian el estado actual de tu colección usando los botones que ya están en la app. Tiempo total estimado: 5 minutos.

A.1 Compilar la última versión

Correr `fuentes\_build\_definitivo.bat`. Asegura que tienes el binario con menú contextual del log + footer en 2 líneas + botón solo verdes + `SetFileAttributesW` para borrado robusto.

A.2 Borrar categorías vacías

Abrir **Categorías** | pulsar **Borrar vacías (N)** | Guardar. Hoy tienes ~6 categorías en 0; los IDs se eliminan del config y las carpetas vacías se borran del disco.

A.3 Compactar numeración

Abrir **Categorías** | pulsar **Compactar** | Guardar. Los IDs quedan 01, 02, 03... consecutivos según orden visual. Soluciona el desorden 22-A + 22-B + saltos de la versión anterior.

A.4 Iterar el descubridor con simulación

Abrir **Descubrir categorías** | marcar candidatos jugosos (chatgpt, codex, generative-ai, claude-code-skill, linux/windows) | pulsar **Simular** | pulsar **Solo verdes** | **Crear y reclasificar**. Solo se crean las que tienen repos reales.

A.5 Configurar Syncthing

En la configuración del folder GitHub en Syncthing | Edit | tab *Ignore Patterns* | agregar `*.tmp`. Evita race conditions con la escritura atómica.

B. Mejoras tecnicas a corto plazo

15 features incrementales agrupadas por area. Cada una es independiente y se puede implementar en orden libre. Las marcadas con (*) son las que mas impacto tienen en el flujo diario.

B1 - Productividad del editor de categorias

* #B1.1 Busqueda incremental por categoria

Problema: Con 25+ categorias en la lista del panel izquierdo, encontrar una especifica requiere scrollear. Cuando tengas 50+ va a ser inviable.

Solucion: Un TextEdit arriba del panel izquierdo que filtra la lista en tiempo real por substring del id o de cualquier keyword/topic_boost. Atajo Ctrl+F para enfocarlo.

Mockup / como se ve:

```
+ - Editor de categorias -----+
| [Q] [claude_____] | <- campo de busqueda
| |
| Categorias (3 de 27) |
| 01-claude-code (47) |
| 13-claude-code-skill (8) |
| 21-claude-ai (5) |
```

Esfuerzo: 30 min | **Beneficio:** Critico cuando >20 categorias

* #B1.2 Drag & drop para reordenar

Problema: Mover una categoria 10 puestos requiere 10 clicks en arriba o abajo. Tedioso para reorganizar.

Solucion: Habilitar drag & drop nativo de egui sobre las filas. Al soltar, la categoria se reposiciona y el state se marca dirty. La ultima posicion sigue siendo el fallback.

Esfuerzo: 1-2h | **Beneficio:** Reordenar categorias 10x mas rapido

* #B1.3 Bulk-edit de keywords y topic_boosts

Problema: Si quieres mover el peso de un topic de 8 a 12 en TODAS las categorias que lo usan, hay que abrir cada una y editar manualmente.

Solucion: Un sub-dialogo Buscar y reemplazar donde escribis topic + nuevo peso, y se aplica a todas las categorias que lo contengan.

Esfuerzo: 1h | **Beneficio:** Refinar pesos sin tedio

B2 - Inteligencia del descubridor

* #B2.1 Preview detallado: que REPOS especificos caerian

Problema: Hoy ves predicted: 5 pero no que 5 repos. Si te equivocas creando una categoria, no puedes validar antes.

Solucion: Click en una fila del descubridor (despues de simular) expande un sub-panel con la lista de repos que caerian en esa categoria, con su descripcion corta. Permite abrir en buscador para ver detalles.

Mockup / como se ve:

```
[V] | 14 | 8 | agents | 11-agents (v)
+> Repos que caerian (8):
- langgraph | Build resilient language agents...
- claude-context | Context management for AI agents
- openhands | OpenHands: Code Less, Make More
- ai-agents-for-... | Hands-on agentic course
```

Esfuerzo: 1h | **Beneficio:** Validar antes de aplicar

* #B2.2 Memoria de descartes

Problema: Cada vez que abris el descubridor te aparecen los mismos candidatos que ya descartaste antes (ai, open-source, etc.) - ruido visual.

Solucion: Un archivo data/descartes_descubridor.json que guarda los topics que marcaste como no me interesa. Los excluimos por default. Boton Mostrar descartados para revisar.

Snippet:

```
{ "topics_descartados": [
  { "topic": "hacktoberfest", "fecha": "2026-05-04", "razon": "evento, no tematica"},
  { "topic": "open-source", "fecha": "2026-05-04", "razon": "demasiado generico" }
]
```

Esfuerzo: 1h | **Beneficio:** Limpia el descubridor de ruido

* #B2.3 Sugerencia de fusiones de categorias existentes

Problema: Algunas categorias terminan siendo casi sinonimos (ej. 03-agentes-y-llms y 11-agents pueden tener mucho overlap).

Solucion: Un analisis post-build que detecta categorias cuyos repos comparten >50% de los topics dominantes. Sugiere fusionarlas con un boton Considera fusionar X y Y.

Esfuerzo: 2h | **Beneficio:** Evitar fragmentacion excesiva

B3 - Sistema de seguimiento de repos

Esta es el area que pediste expandir explicitamente. Hoy el clasificador trata todos los repos por igual; vos los probas y quieres recordar cual funciono, cual descartaste, cual es prioridad. Propongo un sistema de **metadata personal por repo** guardada en `data/repo_notes.json`.

* #B3.1 Estado del repo (probado / descartado / favorito / en uso)

Problema: No hay forma de marcar este repo lo probe y funciono o este lo descarte tras 5 minutos. Cada vez que vuelves a la lista, no recordas cuales ya investigaste.

Solucion: Cada repo puede tener un estado seleccionable de un dropdown corto: **sin probar** (default) | **inspeccionado** | **probado-OK** | **probado-FAIL** | **descartado** | **favorito** | **en uso**. El estado se persiste y se muestra como badge en el buscador.

Snippet:

```
{
  "langgraph": {
    "estado": "probado-OK",
    "fecha_estado": "2026-05-03",
    "probado_version": "v0.2.55"
  },
  "old-experiment": {
    "estado": "descartado",
    "razon_descarte": "no compila en Windows | pesado"
  }
}
```

Esfuerzo: 3h | **Beneficio:** Memoria personal de tu exploracion

* #B3.2 Notas libres por repo (markdown)

Problema: Quieres anotar este lo probe el martes con Python 3.12 y funciono pero requiere CUDA - hoy no hay donde.

Solucion: Campo **notas: String** en `repo_notes.json`, editable desde un sub-dialogo Editar notas en el `buscador.html`. Soporta markdown (renderiza con `pulldown-cmark`). Las notas aparecen al expandir cada card del buscador, debajo de la descripcion oficial.

Mockup / como se ve:

```
+-- buscador.html | langgraph -----+
| Mis notas |
| - Probado 2026-05-03 con Python 3.12 |
| - Necesita: pip install langchain-core |
| - El ejemplo de RAG agentico funciona |
| - TODO: probar con qdrant en lugar de |
| chroma |
+-----+
```

Esfuerzo: 3h | **Beneficio:** Convertir el clasificador en tu cuaderno tecnico

* #B3.3 Carpeta especial _descartados (auto-archivado)

Problema: Repos que probaste y no te sirven siguen ocupando espacio en su categoria, sumando ruido al buscar.

Solucion: Cuando marcas un repo como **descartado**, opcionalmente se mueve a una carpeta especial _descartados/ (con la categoria original como subcarpeta para preservar el origen). En el buscador hay un toggle Mostrar descartados que por default esta OFF.

Snippet:

```
D:\...\GitHub\  
+-- 01-claude-code\  
+-- 03-agentes-y-llms\  
+-- _descartados\  
    +-- 03-agentes-y-llms\  
    |   +-- repo-que-no-funciono\   <- preserva categoria origen  
    +-- 07-multimedia-y-conversion\  
    +-- otro-descartado\  

```

Esfuerzo: 2h | **Beneficio:** Limpieza pasiva sin perder el rastro

* #B3.4 Historial de actividad por repo

Problema: No hay registro de cuando abriste un repo desde el buscador, cuando intentaste correrlo, ni cuando lo viste por ultima vez.

Solucion: Cada interaccion registrada como timestamp en repo_notes.json. Metricas derivadas en el dashboard: top 10 mas visitados, repos sin abrir hace +6 meses, racha de actividad por categoria.

Snippet:

```
{  
  "langgraph": {  
    "abierto_count": 7,  
    "ultima_apertura": "2026-05-04T15:20:00",  
    "primer_clone": "2026-04-12",  
    "intentos_run": [  
      {"fecha": "2026-04-15", "resultado": "fail", "nota": "missing CUDA"},  
      {"fecha": "2026-05-03", "resultado": "ok"}  
    ]  
  }  
}
```

Esfuerzo: 3h | **Beneficio:** Vision analitica de tu coleccion

* #B3.5 Dashboard de actividad personal

Problema: No hay vista panoramica de tu uso del clasificador.

Solucion: Una pagina HTML extra **dashboard.html** generada por el reindex con: distribucion por estado (X probados-OK, Y descartados, Z sin probar), top categorias con mas actividad, repos mas visitados, repos olvidados (sin abrir hace +180 dias). Graficas de barras simples con Chart.js.

Esfuerzo: 3h | **Beneficio:** Auto-conocimiento del flujo

B4 - Quick actions y atajos

* #B4.1 Botones de accion por repo en el buscador

Problema: Para abrir un repo en VS Code, copiar la URL git, o ir a su pagina de GitHub, hay que hacerlo manual desde el explorador.

Solucion: En cada card del buscador.html, agregar 4 botones inline: **Abrir carpeta** | **Abrir en VS Code** | **Abrir en GitHub** | **Copiar git clone**. Los dos primeros usan file:// y custom URL handlers; los otros dos son web/clipboard puros.

Mockup / como se ve:

```
+-- langgraph -----+
| Build resilient language agents as graphs|
| [stack: python, langchain] |
| |
| [abrir] [vscode] [github] [git clone] |
+-----+
```

Esfuerzo: 2h | **Beneficio:** Reducir friccion al explorar repos

* #B4.2 Comparar dos repos lado a lado

Problema: Cuando dos repos hacen casi lo mismo (ej. 2 alternativas RAG), quieres decidir cual mantener. Hoy hay que abrir ambos manualmente y comparar.

Solucion: Checkbox comparar en cada card del buscador (max 2 marcados). Boton Comparar seleccionados abre un modal con descripcion/topics/stats lado-a-lado y resaltado de diferencias.

Esfuerzo: 3h | **Beneficio:** Decision informada sobre duplicados funcionales

* #B4.3 Copiar info al portapapeles en formatos multiples

Problema: Si quieres mencionar el repo en un README propio, en Slack, o en una nota Obsidian, hay que armar la cita a mano.

Solucion: Boton Copiar como... con sub-menu: **markdown link** | **texto plano** | **Obsidian wikilink** | **JSON con metadata**. Cada opcion copia un formato pre-armado.

Snippet:

```
Markdown:    [langgraph](https://github.com/...) - Build resilient...
Texto plano: langgraph (https://github.com/...) - descripcion
Obsidian:    [[langgraph]] - categoria: 03-agentes-y-llms
JSON:        {"name":"langgraph","id":...,"topics":[...]}
```

Esfuerzo: 1.5h | **Beneficio:** Compartir con menos friccion

* #B4.4 Accion Descartar desde el buscador

Problema: Para descartar un repo hoy hay que ir a la GUI nativa o moverlo manual.

Solucion: Boton **Descartar** en cada card. Al pulsar, abre confirm modal y al aceptar marca el repo como **descartado** en repo_notes.json + lo mueve a _descartados/ (si esta habilitada B3.3). Se desvanece de la vista actual con animacion.

Esfuerzo: 1h | **Beneficio:** Limpieza desde el buscador, sin abrir la GUI

B5 - Mantenimiento y observabilidad

* #B5.1 HISTORIA_PROYECTO.txt actualizado

Problema: El changelog del proyecto esta desactualizado desde la fase 2 - faltan 3a, 3b, 3c, 3d, 4 y 5.

Solucion: Regenerar HISTORIA_PROYECTO.txt con el formato existente: secciones por fase + bullets concretos + lecciones aprendidas. Bonus: agregar seccion diseno de datos explicando el rol de cada JSON en data/.

Esfuerzo: 30 min | **Beneficio:** Doc que vale para volver al proyecto en 6 meses

* #B5.2 Refactorizar gui/app.rs en submodulos

Problema: El archivo tiene 3500+ lineas. Cualquier cambio nuevo es dificil de ubicar y los borrows mutables se vuelven hostiles.

Solucion: Mover cada dialogo a su propio modulo bajo gui/dialogs/:

- gui/dialogs/duplicates.rs
- gui/dialogs/reclassify.rs
- gui/dialogs/cats_editor.rs
- gui/dialogs/discover.rs
- gui/dialogs/api_key.rs

Quedaria app.rs con ~800 lineas (estructura principal + workers).

Esfuerzo: 2-3h | **Beneficio:** Mantenibilidad a largo plazo

* #B5.3 Refresh inteligente de GitHub IDs (ETag/If-Modified-Since)

Problema: Cada Refrescar GitHub IDs consume 157 calls a la API aunque nada haya cambiado. Hoy son 67s con PAT (5000/h), pero a 500 repos seria molesto.

Solucion: Guardar el header etag de cada response en repo_ids.json. Proximo refresh manda If-None-Match; si GitHub responde 304, no contamos como request al rate limit y skipeamos parsing.

Esfuerzo: 1.5h | **Beneficio:** Refresh 5-10x mas rapido en colecciones estables

* #B5.4 Tests unitarios del clasificador

Problema: Hoy hay 3 tests en topic_discovery. Si tocas classify_heuristic o reclassify, no hay red de seguridad.

Solucion: Agregar suite con casos sinteticos: repo con topic ambiguo -> categoria correcta, repo sin topics -> cae en fallback, scoring de keywords vs topics, edge cases del descubridor (sinonimos exoticos, pesos extremos). Apuntar a 60-70% de cobertura del modulo classify.

Esfuerzo: 3h | **Beneficio:** Confianza para refactors futuros

C. Features arquitectonicas a futuro

10 features que cambian sustancialmente como pensas la herramienta. Requieren mas disenio previo y posiblemente datos extra. Las propongo como inspiracion, no necesariamente para implementarlas todas.

C1 - Multi-tag y sub-categorias

* #C1.1 Multi-tag ademas de la carpeta unica

Problema: Un repo de RAG con LLM y vector-database hoy tiene que vivir en UNA sola carpeta. Queres filtrar el buscador por TAG arbitrario.

Solucion: Mantener la categoria unica (la carpeta fisica, decision de organizacion). Pero el buscador soporta filtros multi-tag derivados de los topics: el repo aparece bajo cualquier tag que tenga, ademas de su categoria.

Mockup / como se ve:

```
Filtros activos:
categoria: [todas (v)]
tags: [+ rag] [+ vector-db] [+ openai]
+- AND/OR -+
Resultados: 8 repos coinciden con todos los filtros
```

Esfuerzo: 1 dia

* #C1.2 Sub-categorias jerarquicas

Problema: Cuando 01-claude-code tiene 50+ repos se vuelve inmanejable. Aplanar mas con 01-claude-code-skill, 01-claude-code-plugin, etc., genera 30+ buckets.

Solucion: Soportar jerarquia de carpetas fisicas: 01-claude-code/skills/<repo>. El config tiene `parent_id` opcional. El buscador renderiza un arbol expandible.

Snippet:

```
{ "id": "01-claude-code/skills",
  "parent": "01-claude-code",
  "topic_boosts": [["claude-code-skill", 12]] }
```

Esfuerzo: 1.5 dias

C2 - Inteligencia con LLM

* #C2.1 Asistente conversacional sobre tu coleccion

Problema: Cuando quieras saber que repos tengo de RAG con vector store, hoy hay que filtrar a mano.

Solucion: Un panel **Preguntale a Claude** en el buscador. La query se enriquece con el JSON de tu `repos_index.json` (filtrado por relevancia con embeddings) y se manda a Claude API. La respuesta cita repos especificos con links.

Mockup / como se ve:

```
Asistente del clasificador
Tu: Que repos uso para RAG con vector DB local?
Claude: Tienes 3 repos relevantes:
- lightrag - RAG ligero con knowledge graph
- qdrant - vector DB local (Rust)
- langflow - orquestacion visual...
```

Esfuerzo: 2-3 dias | **Beneficio:** Asistente personal sobre tu propia data

* #C2.2 Auto-clasificación con LLM en lugar de heurística

Problema: Algunos repos clasifican mal porque la heurística no captura matices semánticos.

Solución: Modo clasificación con LLM opcional: para repos cuya confianza heurística es <0.5 , mandar (nombre + descripción + topics + 200 chars de README) a Claude Haiku con prompt corto, recibir categoría. Cache para no re-pagar el mismo repo.

Esfuerzo: 2 días | **Beneficio:** Mejor precisión en casos border

* #C2.3 Resumen ejecutivo automático por repo

Problema: Las descripciones oficiales de GitHub son cortas y técnicas. Quieres que hace este repo en 2 líneas en español.

Solución: Botón **Resumir** por repo. Manda README + topics a Claude pidiendo resumen de 2-3 líneas en español que responda para que sirve y cuando lo usaría. Se guarda en repo_notes.json para no re-pagar.

Esfuerzo: 1 día

C3 - Automatizacion y schedule

* #C3.1 Schedule recurrente del refresh

Problema: El refresh de GitHub IDs es manual. Topics y descripciones cambian con el tiempo (autores los editan).

Solucion: Dialogo **Programar tareas** donde configuras: refresh IDs cada 7 dias | reindex automatico cada 24h | alerta si hay >10 repos sin tocar en 6 meses. Se persisten en data/schedule.json. Un loop en el GUI las dispara cuando aplica.

Esfuerzo: 1 dia

* #C3.2 Auto-deteccion de actualizaciones de upstream

Problema: No sabes si tus repos clonados tienen commits nuevos en GitHub respecto a tu copia local.

Solucion: Comparar el SHA del HEAD local vs el ultimo commit del default_branch en GitHub. Mostrar badge **N commits atrasados** en el buscador. Boton git pull en bulk para repos que quieres actualizar.

Esfuerzo: 1.5 dias | **Beneficio:** Mantener tu coleccion viva

* #C3.3 Hook con GitHub Stars

Problema: Si starreas un repo en github.com, no hay forma de saber cuales ya clonaste y cuales falta.

Solucion: Sincronizar tu lista de starred via GitHub API. Mostrar en el dashboard: **clonaste 47 de tus 89 starred | faltan 42**. Boton clonar masivo para los que falten (con confirmacion).

Esfuerzo: 2 dias

C4 - Salud del ecosistema

* #C4.1 Analisis de dependencias y vulnerabilidades

Problema: No sabes cuales de tus repos tienen dependencias con CVEs conocidos.

Solucion: Por cada repo con package.json/requirements.txt/Cargo.toml/etc., correr el auditor nativo (npm audit, pip-audit, cargo-audit). Resumen agregado en el dashboard. Badge rojo en repos con CVE critico.

Esfuerzo: 2-3 dias

* #C4.2 Deteccion de repos abandonados/archivados

Problema: Si un repo upstream fue archivado o no tiene commits hace 2 anos, es senal para reconsiderarlo.

Solucion: Endpoint /repos/<owner>/<repo> incluye archived + pushed_at. Si *archived=true* o *pushed_at < hace 24 meses*, badge **stale** en el buscador. Accion sugerida: marcar como descartado o archivar.

Esfuerzo: 1 dia

* #C4.3 Modo colaborativo / multi-usuario

Problema: Si compartis el folder con un colega o tenes 2 maquinas con perfiles distintos, los archivos se pisan.

Solucion: Soportar multi-perfil: cada usuario tiene su propio repo_notes.json (notas personales) pero comparten categorias.json (taxonomia). Conflict resolution simple por last-write-wins con backup automatico.

Esfuerzo: 3-4 dias

Apendice - Matriz de priorizacion

Recomendacion de orden basada en **impacto en flujo diario** x **esfuerzo de implementacion**. Las que estan en **oleada 1** son las que YO empezaria primero despues del nivel A.

Oleada	Features	Tiempo	Por que
1 - Quick wins	B1.1 busqueda B2.1 preview detallado B4.1 quick actions B4.2 buscador B4.3 descartados B4.4 estados B4.5 salud	~6h	Alta frecuencia de uso. Bajo riesgo. Alto retorno.
2 - Memoria personal	B3.1 estados B3.2 notas B3.3 carpeta_descartados	~8h	Convierte el clasificador en tu cuaderno. Construye seguridad.
3 - Pulido	B1.2 drag & drop B2.2 memoria de descartes B5.1 historial B5.2 refinamiento	~7h	Mejoras esteticas y de mantenibilidad. Hacelas cuando tengas memoria personal.
4 - Analitica	B3.4 historial B3.5 dashboard B5.3 refresh inteligente	~7h	Vista panoramica. Util cuando ya tengas memoria personal.
5 - Inteligencia	C2.1 asistente C2.2 LLM clasificador C2.3 resumen	~5 dias	Big features. Considerar despues de 1-4.
6 - Ecosistema	C1.1 multi-tag C3.x schedule C4.x salud	~10 dias	Cambios arquitectonicos. Solo si la herramienta llega a ser usada.

Estimacion de tiempo total por escenarios

Escenario	Tiempo	Resultado
Solo Nivel A	5 min	Disco prolijo, no hay codigo nuevo
Nivel A + Oleada 1 (B quick wins)	6h	Flujo diario significativamente mejorado
Nivel A + Oleadas 1+2 (memoria)	14h	Clasificador con memoria personal completa
Nivel A + Oleadas 1-4 (todo nivel B)	28h	App completa con analitica y pulido
Todo (A + B + C)	70-90h	Plataforma de gestion de coleccion de repos

Recomendacion final: *empezar con Oleada 1 inmediatamente despues del Nivel A. Son 6 horas de trabajo distribuibles en sesiones de 1-2h, y los 4 features juntos transforman la sensacion de uso. Despues decidir si vale la pena meterse a la Oleada 2 (memoria personal) segun cuanto te termine importando llevar registro detallado de tu exploracion.*